Consumer Groups and Rebalancing

sharing a topic's partitions between consumers

Watch the 4-min video

In one line: consumers with the same group ID split a topic's partitions so each partition is read by one member; scaling past the partition count doesn't help, and every join, leave or timeout triggers a rebalance, so use cooperative rebalancing, static membership and well-tuned timeouts.

1 Scaling that made it worse

Your consumers can't keep up and lag is climbing. You scale from four instances to twelve. Lag keeps growing, and every deploy makes things worse as the whole group stops to **rebalance**. To fix it you need to know how consumer groups divide the work.

2 How a consumer group works

Consumers that share a **group ID** form a group. The broker's **group coordinator** assigns each partition to exactly one member, and each member reads its partitions and commits offsets for the group. Another group reading the same topic gets its own full copy of the stream. Inside a group, the **partition count caps parallelism**.

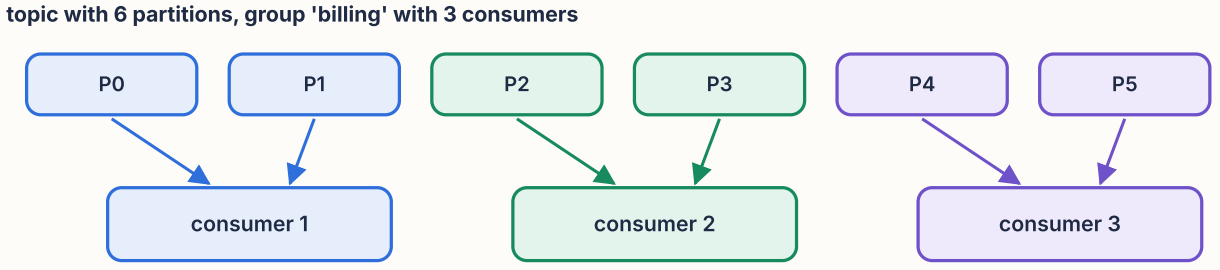


Figure 1. Each partition belongs to one consumer of the group; one consumer can own several partitions.

24 partitions, 12 consumers → 2 each ✓

24 partitions, 30 consumers → 6 consumers sit idle ✕

a partition is read by at most one consumer in a group, so partitions cap parallelism

Figure 2. Extra consumers beyond the partition count do nothing.

3 Rebalancing

When a member joins, leaves, crashes, or misses its deadline (no heartbeat within `session.timeout.ms`, or no poll within `max.poll.interval.ms`), the group **rebalances**: partitions are reassigned. With the classic **eager** protocol, every consumer stops, gives up all its partitions and waits for the new assignment. A rolling deploy of 12 instances can cause a dozen stop-the-world pauses.

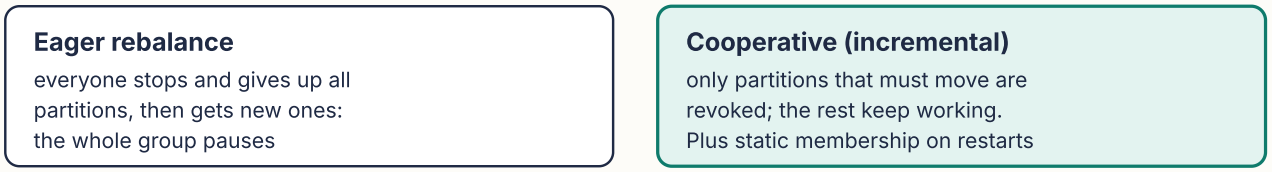


Figure 3. Eager vs cooperative rebalancing.

Modern fixes: the **cooperative sticky** assignor moves only the partitions that need to move; **static membership** (`group.instance.id`) lets a restarting instance come back without triggering a rebalance if it returns within the session timeout; and Kafka 4's new consumer group protocol moves assignment to the broker and makes it incremental.

WORKED EXAMPLE: FOUR CONSUMERS, TWENTY-FOUR PARTITIONS

A group of four consumers reads a topic with 24 partitions, six each, and lag is growing. Scaling to 12 instances gives two partitions each and roughly triples throughput, but going to 30 would leave six idle. Each processing batch takes up to 6 minutes because of a slow downstream call, longer than the 5-minute `max.poll.interval.ms`, so consumers keep getting kicked out, causing rebalance storms. The fix: `max.poll.records` from 500 to 100, the cooperative sticky assignor, static membership for Kubernetes pods, and a separate thread pool for the slow call. Lag drains within the hour.

COMMON MISTAKES

- More consumers than partitions, expecting more throughput.
- Slow processing in the poll loop, exceeding max.poll.interval.ms.
- Eager rebalancing with frequent deploys or autoscaling.
- Two unrelated services sharing a group ID by accident.

4 Key terms

Consumer group: consumers sharing a group ID and the partitions

Rebalance: reassigning partitions when membership changes

Static membership: stable member ID avoids rebalances on restart

Group coordinator: broker that manages membership and offsets

Cooperative sticky: incremental assignor that moves few partitions

Consumer lag: latest offset minus committed offset

5 Interview questions

Q1. How do consumer groups share work? BASIC

Each partition is assigned to exactly one consumer in the group; consumers can own several partitions. Different groups each read the whole topic.

Q2. What triggers a rebalance? BASIC

A consumer joining or leaving, a crash, missed heartbeats beyond the session timeout, exceeding max.poll.interval.ms, or a change in subscribed partitions.

Q3. You doubled consumers but lag didn't drop. Why? SCENARIO

There may be no more partitions than consumers, the bottleneck may be downstream, or rebalances keep pausing the group. Check partition count, processing time and rebalance frequency.

Q4. Every Kubernetes deploy pauses consumption for a minute. Fix it. SCENARIO

Use the cooperative sticky assignor, static membership with group.instance.id, and graceful shutdown that leaves the group cleanly.

Q5. What's the difference between session.timeout.ms and max.poll.interval.ms? TRICKY

Session timeout covers heartbeats from the background thread (is the process alive?); max poll interval covers progress (is the poll loop still running?).

Q6. Can two consumers in one group read the same partition at once? TRICKY

Not normally; during an eager rebalance there's a gap, and with cooperative rebalancing a partition is revoked before it's reassigned, so ownership doesn't overlap.

Next: SD-I-006 · Kafka Replication and Durability backendroadmap.com